Universidad Nacional Autónoma de México

Facultad de Ingeniería

Sistemas Operativos

Unidades de Procesamiento Neuronal

Integrantes:

Brena de León Victor Javier

No. de Cuenta: 321117524

Cruz Manriquez Lizbeth

No. de Cuenta: 424128454

Dr. Gunnar Eyal Wolf Iszaevich

Grupo: 7

Fecha de entrega: **05/03/2026**

Semestre: **2026-2**

Índice de contenido

1. Introducción **3**
1. Glosario de Términos **4**
2. Fundamentos de las Redes Neuronales y Hardware **6**
2.1 Inspiración biológica vs. Realidad matemática 6
2.2 Operaciones matemáticas predominantes 6
2.3 Tipos de datos y precisión: Salto de FP32 a INT8 7
2.3.1. Comparativa de estructuras de datos: 7
2.3.2. La precisión en los datos 7
2.3.3. Ventajas de utilización de menos bits 7
3. Arquitectura de un NPU **8**
3.1 Características principales 8
3.2 Componentes principales 9
3.2.1 Núcleos tensoriales 9
3.2.2 Arquitectura de matriz sistólica 9
3.2.3. Memoria de alto ancho de banda 9
3.2.4. Interfaces Externas 9
3.3. Cuantización: De FP32 a INT8 10
3.4. El flujo de ejecución. 11
A. Compilación Offline 11
B. Intervención del Controlador 11
C. Uso del DMA (Acceso Directo a Memoria) 11
D. Ejecución Autónoma 12
4. Interacción Sistema-NPU **12**
4.1 Ciclo de vida de una tarea de IA: 12
4.2 Herramientas de compilación y despliegue: 13
5. Aplicaciones y Casos de Uso **13**
5.1 Algoritmos acelerados por hardware 13
a) Redes Neuronales Convolucionales (CNN) 13
b) Redes Neuronales Recurrentes (RNN) 13
c) Transformers 13
d) Perceptrón Multicapa (MLP) 14
5.2 Inferencia en el Edge y Beneficios industriales 14
6. Comentarios finales **14**
7. Referencias Bibliográficas **16**

1. Introducción

El objetivo de este documento es analizar la arquitectura y funcionamiento de las NPU, explicando su estructura interna, semejanza con las redes neuronales biológicas, algoritmos que ejecutan y su impacto en la computación moderna.

En los últimos años, el crecimiento acelerado de lo que popularmente se denomina Inteligencia Artificial (IA)(técnicamente, modelos de aprendizaje profundo) ha generado nuevas exigencias en el diseño de hardware especializado. Procesadores tradicionales como la Unidad Central de Procesamiento (CPU) y la Unidad de Procesamiento Gráfico (GPU) han sido fundamentales en la evolución de la informática; sin embargo, presentan limitaciones cuando ejecutan cargas de trabajo intensivas propias de redes neuronales profundas, especialmente en términos de eficiencia energética y latencia.

Como respuesta a estas demandas surgieron las Unidades de Procesamiento Neuronal (NPU) siendo este un microprocesador especializado diseñado para acelerar operaciones matemáticas asociadas con redes neuronales artificiales y modelos de aprendizaje profundo. El cual está principalmente diseñado para acelerar la fase de inferencia en modelos de inteligencia artificial, especialmente en redes neuronales convolucionales (CNN) y redes neuronales recurrentes (RNN).

Comparación de los procesadores usado en modelos de aprendizaje

Característica	CPU	GPU	NPU
Propósito	Procesamiento general	Procesamiento gráfico y paralelo	Procesamiento especializado en IA
Tipo de arquitectura	Secuencial con pocos núcleos	Masivamente paralela	Paralelismo optimizado para tensores
Núcleos	2–64	Cientos o miles	Miles de unidades MAC
Precisión típica	FP64, FP32	FP32, FP16	INT8, FP16
Consumo energético	Moderado	Alto	Bajo
Uso principal	Sistema operativo, aplicaciones	Gráficos, entrenamiento IA	Inferencia IA
Eficiencia en IA	Baja	Alta	Muy alta
Movimiento de datos	Alto	Alto	Optimizado

1. Glosario de Términos

Término	Definición	Referencia (Pág.)
Arquitectura Sistólica	Estructura de procesamiento donde los datos fluyen rítmicamente entre unidades de cálculo, minimizando el acceso a memoria externa.	https://en.wikipedia.org/wiki/Systolic_array
Backpropagation	Algoritmo que calcula gradientes para actualizar los pesos de una red neuronal durante la fase de entrenamiento.	https://www.geeksforgeeks.org/backpropagation-in-convolutional-neural-networks/
CNN	*(Convolutional Neural Network)* Red neuronal diseñada para procesar datos con estructura espacial, como imágenes.	https://en.wikipedia.org/wiki/Convolutional_neural_network
Edge Computing	Procesamiento de datos cerca del dispositivo que los genera (localmente), reduciendo la latencia y la dependencia de la nube.	https://www.ibm.com/topics/edge-computing
Entrenamiento	Proceso de ajuste de los parámetros (pesos) en una red neuronal mediante algoritmos de optimización.	https://www.sciencedirect.com/topics/engineering/neural-network-training
Forward Propagation	Etapa en la que los datos de entrada atraviesan las capas de la red neuronal para generar una salida o predicción.	https://d2l.ai/chapter_multilayer-perceptrons/backprop.html

Inferencia	Fase en la que un modelo ya entrenado realiza predicciones o clasificaciones sobre datos nuevos en tiempo real.	https://www.ibm.com/think/topics/ai-inference
INT8 / FP16	Formatos numéricos de baja precisión (enteros de 8 bits o punto flotante de 16 bits) que optimizan el consumo energético y la velocidad.	https://en.wikipedia.org/wiki/Half-precision_floating-point_format https://en.wikipedia.org/wiki/Integer_(computer_science)
IoT	*(Internet of Things)* Red de dispositivos físicos (sensores, cámaras, wearables) que recopilan e intercambian datos a través de internet.	https://www.ibm.com/think/topics/internet-of-things
MAC	*(Multiply-Accumulate)* Operación aritmética fundamental ($A = A + (B \times C)$) que las redes neuronales ejecutan millones de veces por segundo.	https://en.wikipedia.org/wiki/Multiply%E2%80%93accumulate_operation
RNN	*(Recurrent Neural Network)* Red neuronal que procesa información secuencial (como audio o series de tiempo) manteniendo memoria de estados previos.	https://www-ibm-com.translate.goog/think/topics/recurrent-neural-networks?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc
SoC	*(System on Chip)* Circuito integrado que integra en un solo chip la CPU, GPU, NPU y otros controladores del sistema.	https://www.baesystems.com/en-us/definition/what-is-a-system-on-a-chip

Transformer	Arquitectura basada en mecanismos de atención que permite el procesamiento en paralelo de grandes volúmenes de datos (Ceva, 2023).	https://www.ceva-ip.com/blog/transformer-models-and-npu-ip-co-optimized-for-the-edge/

2. Fundamentos de las Redes Neuronales y Hardware

Para entender por qué el hardware requiere una especialización tan profunda, es necesario primero analizar la naturaleza matemática de los modelos que ejecuta. El diseño de una NPU surge de intentar traducir la eficiencia del procesamiento biológico a estructuras lógicas de silicio.

2.1 Inspiración biológica vs. Realidad matemática

Las NPU están inspiradas en el modelo matemático de las redes neuronales biológicas, donde múltiples nodos interconectados ajustan pesos para optimizar decisiones.

Lo que hace que NPU sea parecida y sobre todo diseñada a las funciones del cerebro son:

Cerebro Humano	NPU
Neuronas	Nodos artificiales
Sinapsis	Pesos
Plasticidad	Ajuste de pesos
Aprendizaje	Entrenamiento

Las NPU no replican biológicamente el cerebro, pero sí implementan su modelo matemático basado en redes interconectadas con pesos ajustables.

2.2 Operaciones matemáticas predominantes

A diferencia de los procesadores convencionales, estas unidades priorizan el procesamiento paralelo masivo y el uso de aritmética de baja precisión.

De acuerdo con IBM (2024), una NPU está optimizada para manejar operaciones matemáticas asociadas con redes neuronales, particularmente multiplicaciones matriciales y procesamiento tensorial, reduciendo significativamente el consumo energético en comparación con CPU y GPU. Por lo que es importante analizar estas operaciones que predominan.

a) Unidades MAC (Multiply-Accumulate)
Las operaciones básicas de una red neuronal son:

$$Y = (W*X) + b$$

Donde:
W = pesos
X = entradas

b = sesgo
Las unidades MAC realizan multiplicación y suma en un solo ciclo, acelerando:

- Multiplicación de matrices
- Producto punto
- Convoluciones

Estas operaciones representan más del 90% del cálculo en redes profundas (Sze et al., 2017).

2.3 Tipos de datos y precisión: Salto de FP32 a INT8

Para entender la eficiencia de una NPU, debemos analizar cómo se representan los números en el hardware. Una NPU trabaja principalmente con datos cuantificados en 8 y 16 bits, lo que permite reducir el consumo energético y aumentar la velocidad de procesamiento. Los pesos del modelo suelen manejarse en formato de 8 bits (INT8), optimizando el rendimiento frente a los formatos tradicionales de alta precisión como FP32.

2.3.1. Comparativa de estructuras de datos:

- FP32 (Punto Flotante de 32 bits): Es un estándar en CPUs y GPUs durante el entrenamiento. Ofrece una resolución altísima (representa números muy pequeños o muy grandes de forma precisa). La desventaja, cada número ocupa mucho espacio en memoria y requiere circuitos complejos para ser procesado.
- FP16 (Punto Flotante de 16 bits): Mantiene la capacidad de manejar decimales pero reduce a la mitad el tamaño del dato. Se usa en GPUs modernas para acelerar el entrenamiento sin perder demasiada calidad.
- INT8 (Entero de 8 bits): Este formato es el mas compacto y es el estándar de las NPUs. Para este formato ya no hay decimales y los números se mapean a un rango entero (-128 a 127).

2.3.2. La precisión en los datos

En la GPU (Alta Precisión): Durante el entrenamiento, el modelo necesita FP32 porque está aprendiendo y los ajustes en los pesos son mínimos.

En la NPU (Baja Precisión): En la fase de inferencia, el modelo ya aprendió. No se necesita tanta resolución para saber si una imagen es cierta figura u otra. Con INT8, la NPU puede meter 4 operaciones en el espacio donde la GPU metería solo 1 de FP32.

2.3.3. Ventajas de utilización de menos bits

- Eficiencia Energética: Mover 8 bits desde la memoria RAM el procesador es mucho mejor. Como menciona Mdpi (2025), esto permite ahorros de energía de hasta el 70%.
- Ancho de Banda: Permite cuatro veces el envío de información interna.
- Velocidad (Paralelismo):. En el mismo espacio de silicio donde cabe un multiplicador de 32 bits, podemos poner muchos más de 8 bits.

Se hizo mención que la NPU su uso está principalmente dirigido para realizar inferencia, a diferencia de la GPU que es utilizada para que el modelo “aprenda”, el modelo diseñado para la GPU(FP32) solo puede ser interpretado por la GPU, por lo que para poder hacer inferencia con el modelo es necesario realizar una cuantización para que no haya incompatibilidad con la NPU(INT8), por lo que los valores de punto flotante se mapean a rangos de 8 bits, y aunque se pierde resolución la NPU recompensa está perdida con su paralelismo masivo.

3. Arquitectura de un NPU

Una vez comprendida la base matemática, es necesario analizar cómo se organiza físicamente el hardware para generar la eficiencia energética ya planteada.

Según Hennessy & Patterson (2019), la especialización arquitectónica mejora significativamente la eficiencia energética cuando el hardware se diseña para cargas específicas.

3.1 Características principales

Antes de analizar sus componentes, es necesario entender los pilares que definen a una NPU frente a otros procesadores.

- Arquitectura optimizada para operaciones tensoriales.

Las NPU incorporan hardware que está especialmente diseñado para realizar operaciones tensoriales y matriciales, que son las bases de los algoritmos de aprendizaje profundo. Esto significa que elementos como los mecanismos de multiplicación y acumulación (MAC) y los núcleos de cálculo de tensores están integrados de forma nativa, permitiendo realizar estas operaciones con mucha más eficiencia que con un CPU tradicional. (IBM)

Cabe destacar que al maximizar el paralelismo interno de los tensores es clave para disminuir los accesos a memoria y aumentar el rendimiento global de estos cálculos especializados. (ScienceDirect, 2023).

- Procesamiento paralelo masivo.

Menciona IBM, la capacidad para descomponer tareas en unidades paralelas de proceso. Mientras que las CPU trabajan de forma mayormente secuencial, las NPU ejecutan miles de operaciones al mismo tiempo, permitiendo que tareas como inferencia se realicen con alta velocidad.

- Alta eficiencia energética.

El consumo energético es una prioridad crítica. Al poseer operaciones de baja precisión (como INT8 en lugar de FP32) reduce el uso de energía considerablemente. A partir de ellos se demostró que consume un 35–70% menos energía que los GPU para tareas de inferencia. (Mdpi, 2025).

- Baja Latencia y Edge AI

El procesamiento independiente permite respuestas en tiempo real (visión artificial, robótica) sin depender de servidores externos (Lenovo, 2025).

Esta eficiencia se logra mediante la integración en sistemas SoC (System on Chip) o como aceleradores dedicados.
Pueden integrarse de dos formas principales:

1. Integradas en un System-on-Chip (SoC) junto con CPU, GPU y otros módulos (como en teléfonos inteligentes y cámaras inteligentes), lo que permite que todo el procesamiento ocurra en un solo chip físico con menores costos y menor consumo.

2. Como acelerador independiente en servidores o equipos de alto rendimiento, conectadas directamente a la placa base o a través de interfaces especializadas, proporcionando procesamiento de IA de alto volumen en centros de datos. (IBM).

3.2 Componentes principales

3.2.1 Núcleos tensoriales

Los tensores son estructuras matemáticas multidimensionales (matrices extendidas).
Las NPU incluyen núcleos especializados para:

- Operaciones matriciales de gran escala
- Multiplicaciones tensoriales
- Cálculo de atención en modelos Transformer

3.2.2 Arquitectura de matriz sistólica

Una arquitectura sistólica permite que los datos fluyan entre unidades de cálculo sin regresar constantemente a la memoria principal.
Ventajas:

- Reduce consumo energético
- Minimiza latencia
- Mejora reutilización de datos

3.2.3. Memoria de alto ancho de banda

El acceso a datos es un cuello de botella en los modelos de aprendizaje. Por esta razón las NPU incorporan:

- Buffers locales y memoria *on-chip* (SRAM)
- Memoria on-chip
- Reducción de accesos DRAM
- El uso de un controlador DMA (Direct Memory Access) permite transferir pesos y mapas de características (IFM/OFM) sin intervención de la CPU, optimizando el ancho de banda.

3.2.4. Interfaces Externas

La NPU utiliza el estándar AMBA y AMBA 5 AXI, lo que incluye es un gestor de lectura/escritura (M0) y un gestor de solo lectura (M1). Esto permite que la NPU realice múltiples transacciones simultáneas hacia memoria externa, aumentando el paralelismo.

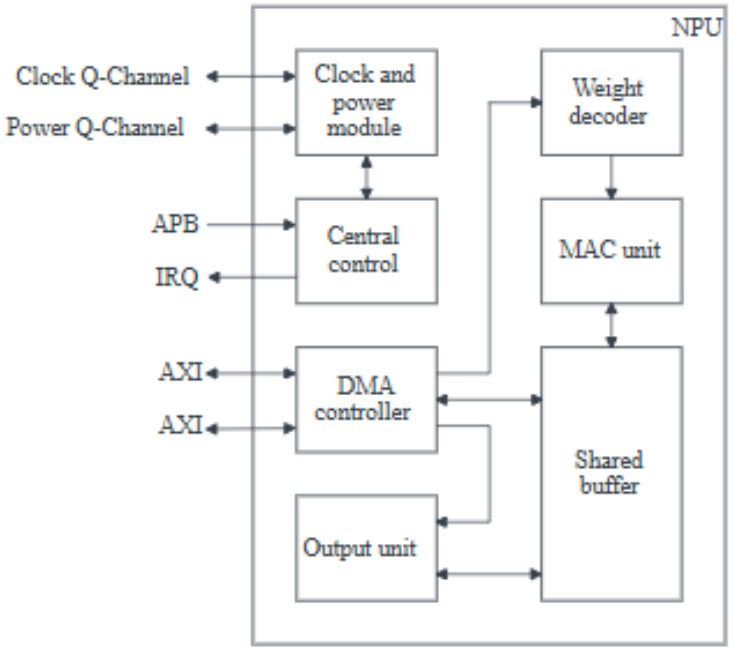

Figura 1. Estructura interna de un NPU

3.3. Cuantización: De FP32 a INT8

La NPU está principalmente dirigido para realizar inferencia, a diferencia de la GPU que es utilizada para que el modelo “aprenda”, el modelo diseñado para la GPU(FP32) no es compatible con la NPU(INT8), por lo que para poder hacer inferencia con el modelo es necesario realizar una cuantización para que no haya incompatibilidad.

La cuantización es el proceso de adaptar un modelo de alta resolución (FP32) a un formato de menor precisión (INT8). Al reducir el tamaño de los pesos a 8 bits, se logra:

1. Reducción de memoria: El modelo ocupa 1/4 del espacio original.
2. Menor consumo: Mover 8 bits por el bus de datos gasta menos energía que mover 32.
3. Mayor velocidad: El hardware puede ejecutar más operaciones por ciclo al ser los multiplicadores de 8 bits físicamente más sencillos.

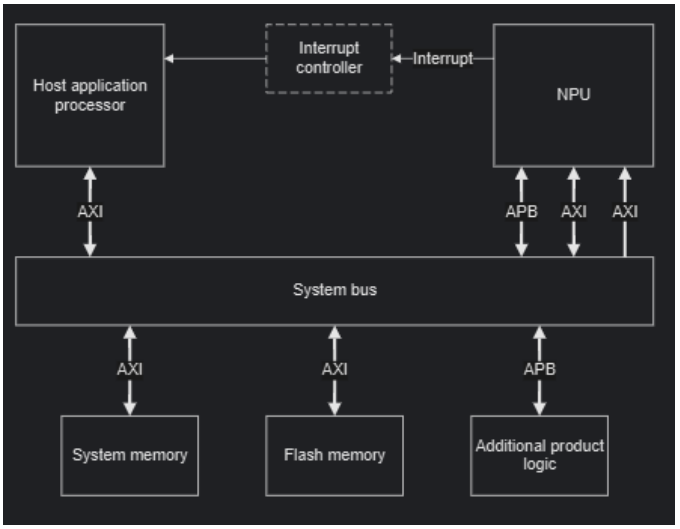

Figura 2. Muestra una configuración del sistema típico para la NPU, (arm developer)

3.4. El flujo de ejecución.

Finalmente, el proceso de ejecución descrito por arm Developer se resume en:

A. Compilación Offline

Se utiliza un compilador de código abierto que transforma la red neuronal en un flujo de comandos.
Este flujo contiene:

- Operaciones de la red
- Organización de memoria
- Parámetros cuantificados

B. Intervención del Controlador

Cuando la aplicación quiere ejecutar la red:
Llama al driver de la NPU. El driver indica a la NPU dónde se encuentra el flujo de comandos en memoria. Se inicia la ejecución y el procesamiento es gestionado internamente por el Control Central (CC) de la NPU.

C. Uso del DMA (Acceso Directo a Memoria)

La NPU integra un controlador DMA, que permite:

- Leer pesos
- Leer sesgos
- Leer factores de escala
- Transferir mapas de características de entrada (IFM)
- Transferir mapas de salida (OFM)
- Guardar datos intermedios
- Todo esto se hace sin intervención constante de la CPU, lo que reduce la latencia.

D. Ejecución Autónoma

Una vez iniciado el proceso la NPU ejecuta los operadores descritos en el flujo de comandos. Utiliza un búfer de trabajo en SRAM para almacenar temporalmente datos en proceso, el DMA gestiona las transferencias hacia y desde memoria externa. Cuando termina la inferencia, la NPU envía una solicitud de interrupción (IRQ) al controlador indicando que la tarea concluyó.

I. Diagrama de flujo de como trabaja una NPU (arm Developer)

[1] Modelo IA (CNN/RNN)
↓
[2] Compilador Offline
↓
Genera Flujo de Comandos
↓
[3] Aplicación → Driver NPU
↓
[4] NPU recibe puntero al flujo
↓
[5] DMA transfiere:
- Pesos
- Sesgos
- Escalas
- IFM / OFM
↓
[6] Ejecución en SRAM interna
↓
[7] Resultado
↓
[8] IRQ → Fin de proceso

4. Interacción Sistema-NPU

Tras comprender la arquitectura física, es necesario analizar el modelo de interacción entre el software y el hardware para ejecutar una tarea de aprendizaje profundo.

4.1 Ciclo de vida de una tarea de IA:

La ejecución de una red neuronal en la NPU requiere una coordinación con la CPU. Este ciclo se divide en tres etapas fundamentales:

1: Carga del modelo
El CPU transfiere a la memoria accesible por la NPU un modelo previamente entrenado (con pesos ya ajustados). En esta fase, los datos de entrada se colocan en buffers específicos gestionados por el DMA.

2: Inferencia

La NPU ejecuta y toma el control
Realiza las multiplicaciones de matrices y aplica las funciones de activación (ReLU o Sigmoid) mediante Forward Propagation (propagación hacia adelante).
Durante este tiempo, la NPU gestiona sus propias interrupciones para no saturar al procesador principal.

3: Salida
Entrega clasificación, predicción o decisión en tiempo real.

4.2 Herramientas de compilación y despliegue:

TensorFlow Lite
Durante la ejecución, el TensorFlow Lite carga un archivo flatbuffer, este archivo contiene el flujo de comandos generado previamente y puede ejecutarse toda la red como si fuera un único operador dentro del acelerador, lo cual, permite que la inferencia ocurra completamente en hardware especializado.

5. Aplicaciones y Casos de Uso

La versatilidad de las NPU permite acelerar los diversos algoritmos de aprendizaje profundo descritos por Goodfellow et al. (2016), adaptándose a las necesidades específicas de cada industria.

5.1 Algoritmos acelerados por hardware

a) Redes Neuronales Convolucionales (CNN)

Aplicaciones:
Visión artificial
Reconocimiento facial
Detección de objetos

Operación principal:
Convolución (multiplicación matricial especializada)

b) Redes Neuronales Recurrentes (RNN)

Aplicaciones:
Procesamiento de lenguaje natural
Reconocimiento de voz
Procesan información secuencial.

c) Transformers

Base de modelos de lenguaje actuales.

Utilizan:
Mecanismo de atención (Self-Attention)
Operaciones matriciales masivas

d) Perceptrón Multicapa (MLP)

Estructura básica:
Capa de entrada
Capas ocultas
Capa de salida

Es la base matemática de redes profundas.

5.2 Inferencia en el Edge y Beneficios industriales

La NPU destaca en la inferencia y no en el entrenamiento. Esto se debe a que la inferencia en dispositivos locales (Edge Computing) prioriza la eficiencia térmica y la baja latencia.

Al procesar los datos directamente en el dispositivo (teléfonos, robots, sensores IoT), se eliminan los tiempos de espera de envío a la nube y se protege la privacidad del usuario. Esta capacidad ha permitido el auge de aplicaciones críticas como:

- IA generativa
- Reconocimiento de voz
- Procesamiento de lenguaje natural
- Vehículos autónomos
- Robótica
- IoT
- Edge Computing
- Dispositivos móviles

6. Comentarios finales

El desarrollo de las Unidades de Procesamiento Neuronal (NPU) se puede comprender mejor si se analizan las limitaciones históricas de la arquitectura de computadoras tradicional. Durante décadas, la mayoría de los sistemas informáticos se han basado en la arquitectura de Von Neumann, un modelo en el que tanto las instrucciones como los datos se almacenan en la misma memoria y se transfieren hacia la CPU a través de un bus común. Este diseño, aunque flexible y ampliamente adoptado, introduce una limitación conocida como cuello de botella de Von Neumann, en la cual la velocidad de transferencia entre la memoria y el procesador es significativamente menor que la velocidad de procesamiento de la CPU. Como consecuencia, el procesador frecuentemente debe esperar a que los datos lleguen desde la memoria antes de continuar con la ejecución de instrucciones, reduciendo la eficiencia general del sistema (Wikipedia, 2024; TechTarget, 2023).

Este problema se vuelve especialmente crítico en aplicaciones modernas de inteligencia artificial. Los modelos de aprendizaje profundo manejan enormes volúmenes de parámetros y requieren millones o incluso miles de millones de operaciones matriciales. En este contexto, el mayor consumo energético y tiempo de ejecución no se debe únicamente al cálculo, sino al movimiento constante de datos entre memoria y unidades de procesamiento.

Investigaciones recientes han mostrado que, en muchas cargas de trabajo de inteligencia artificial, gran parte de la energía del sistema se emplea en transferir datos entre memoria y procesador, lo que agrava aún más el problema del cuello de botella (IBM Research, 2024).

Durante muchos años, el incremento del rendimiento computacional se logró gracias a la Ley de Moore, propuesta por Gordon Moore en 1965, la cual establecía que el número de transistores en un circuito integrado se duplicaría aproximadamente cada dos años. Esta tendencia permitió el desarrollo de procesadores cada vez más potentes, favoreciendo el crecimiento de la informática y de aplicaciones complejas como el aprendizaje automático. Sin embargo, en las últimas décadas se ha observado una desaceleración en esta tendencia debido a limitaciones físicas en la miniaturización de los transistores y al incremento del consumo energético. Como resultado, la industria ha comenzado a explorar nuevas estrategias para mejorar el rendimiento, entre ellas el diseño de aceleradores especializados para tareas específicas.

En este contexto surgen las NPU (Neural Processing Units), que representan una evolución en la arquitectura de hardware orientada específicamente al procesamiento de redes neuronales. Sus antecedentes pueden encontrarse en el uso de GPU para aprendizaje profundo y en arquitecturas de procesamiento paralelo como los arreglos sistólicos, diseñados para ejecutar operaciones matriciales de forma eficiente mediante el flujo continuo de datos entre unidades de cálculo. Estas arquitecturas permiten reducir la dependencia de la memoria externa y mejorar la reutilización de datos, lo que contribuye a disminuir el impacto del cuello de botella de memoria (Sze et al., 2017).

Las NPU integran miles de unidades de cálculo especializadas —principalmente unidades MAC (Multiply–Accumulate)— capaces de ejecutar operaciones tensoriales de forma masiva y con formatos de baja precisión como INT8 o FP16. Gracias a esta especialización, estas unidades pueden ejecutar tareas de inferencia con una eficiencia energética significativamente mayor que las CPU y GPU tradicionales. Además, al incorporar memorias locales, buffers y arquitecturas de flujo de datos optimizadas, las NPU reducen considerablemente el tráfico hacia la memoria principal, mitigando parcialmente el problema estructural de la arquitectura de Von Neumann.

En consecuencia, las NPU representan una respuesta arquitectónica a las nuevas demandas de la computación moderna. Al combinar paralelismo masivo, optimización del flujo de datos y especialización en operaciones tensoriales, estas unidades permiten ejecutar modelos de inteligencia artificial con menor latencia y mayor eficiencia energética, lo que resulta fundamental en aplicaciones de edge computing, dispositivos móviles, sistemas de visión artificial y plataformas IoT.

7. Referencias Bibliográficas

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press. https://www.deeplearningbook.org

Hennessy, J. L., & Patterson, D. A. (2019). Computer architecture: A quantitative approach (6th ed.). Morgan Kaufmann.

IBM. (2024). What is a neural processing unit (NPU)? https://www.ibm.com/think/topics/neural-processing-unit

Sze, V., Chen, Y. H., Yang, T. J., & Emer, J. S. (2017). Efficient processing of deep neural networks: A tutorial and survey. Proceedings of the IEEE, 105(12), 2295–2329. https://ieeexplore.ieee.org/document/8114708

Toryano, Olga (2025). NPU (Unidad de Procesamiento Neuronal): la clave para llevar la Inteligencia Artificial al edge computing industrial. https://eosiberica.es/npu-unidad-de-procesamiento-neuronal-la-clave-para-llevar-la-inteligencia-artificial-al-edge-computing-industrial/?srsltid=AfmBOopauRIqgKVNy1U-fx4u21pZ9-yiKcqJg9lm_lTbPmK223KZFfjh

Ceva (2023). Transformer Models and NPU IP Co-Optimized for the Edge. https://www.ceva-ip.com/blog/transformer-models-and-npu-ip-co-optimized-for-the-edge/

Mdpi. (2025). Performance and Efficiency Gains of NPU-Based Servers over GPUs for AI Model Inference. Systems, 13(9), 797. https://www.mdpi.com/2079-8954/13/9/797

ScienceDirect. (2023). Tensor slicing and optimization for multicore NPUs. Journal of Parallel and Distributed Computing, 175, 66–79. https://doi.org/10.1016/j.jpdc.2022.12.008

Lenovo US. (2025). Understanding Neural Processing Units (NPUs). https://www.lenovo.com/us/en/knowledgebase/understanding-neural-processing-units-npus/

arm Developer. (S/F) Descripción de la unidad de procesamiento neuronal. https://developer.arm.com/documentation/102420/0200/Neural-processing-unit-introduction/Description-of-the-neural-processing-unit

NVIDIA. (2023). *Introduction to INT8 Quantization of Deep Learning Models*. NVIDIA Technical Documentation. https://developer.nvidia.com/blog/achieving-fp32-accuracy-for-int8-inference-using-quantization-aware-training-with-tensorrt/

IBM Research. (2024). Why the von Neumann architecture is impeding the power of AI computing. https://research.ibm.com/blog/why-von-neumann-architecture-is-impeding-the-power-of-ai-computing

Sze, V., Chen, Y., Yang, T., & Emer, J. (2017). Efficient processing of deep neural networks: A tutorial and survey. arXiv. j/https://semiwiki.com/wp-content/uploads/2018/03/2017_pieee_dnn.pdf

TechTarget. (2023). Von Neumann bottleneck definition. https://www.techtarget.com/whatis/definition/von-Neumann-bottleneck

Wikipedia. (2024). Von Neumann architecture. https://en.wikipedia.org/wiki/Von_Neumann_architecture

Wikipedia. (2024). Arquitectura de Von Neumann. https://es.wikipedia.org/wiki/Arquitectura_de_Von_Neumann